

UnifAI - Technical Documentation

Audience: developers, architects, integrators

Scope: architecture, modules, data model, HTTP surface, plugins, Browser AI internals, UI, security model

Baseline: repo commit **16d4c3e**

1. System overview

```

##### UnifAI Go binary (fasthttp) #####
/v1/* inference (OpenAI-compatible) ■ AI apps #####
/openai|anthropic|genai|bedrock|cohere|cursor|litellm|langchain|pydanticai pass-through ■ /api/* dashboard,
governance, observability, Browser AI control plane ■ Admin UI ##### ■ /ws, /v1/realtime WebSocket +
realtime transports ■ (embedded) ■ /health, /robots.txt, /metrics, /pprof (dev) ■ #####
##### ■ ■ ConfigStore
(Postgres) LogsStore (Postgres) users, roles, VKs, keys, LLM logs, MCP logs, Browser AI logs, providers,
settings search logs, dashboard aggregates ■ ##### ■ UnifAI core (Go) ■ →
provider SDKs/HTTP → ~28 LLM vendors ■ plugins, MCP client ■ ##### Guard fleet
(endpoint agents + network proxy) ■ HTTPS ■ /api/browser-ai/* ■ ■ Postgres browser_ai *

```

Two data planes exist in the same binary:

1. **Data plane** (`/v1/*`, pass-through integrations) - request/response proxying to AI providers, with governance, guardrails, caching, logging, tracing.
2. **Control plane** (`/api/*`) - the dashboard: configuration, RBAC, observability and the whole Browser AI guardrail control plane.

2. Technology stack

Layer	Technology	Source of truth
Gateway	Go 1.26.4, valyala/fasthttp , fasthttp/router , automaxprocs	go.work , transports/unifai-http/main.go
Persistence	GORM; Postgres / MySQL / SQLite (CGO go-sqlite3 , sqlite_static)	framework/configstore , framework/logstore
Provider SDKs	Vendor HTTP clients + official SDKs	core/providers/*
Plugins	Go interfaces, optional .so dynamic loading (SharedObjectPluginLoader)	plugins/ , server/plugins.go
MCP	Model Context Protocol client/server, OAuth2 sessions	core/mcp , handlers/mcp*.go
Frontend	React 19, TypeScript 5.9, Vite 8, TanStack Router/Table, Redux Toolkit, Radix UI, Tailwind 4, Monaco, Recharts, Zod	ui/package.json
Guard agent	Python 3.11+, mitmproxy addon, PyInstaller, Inno Setup (Windows), pkg/dmg scripting (macOS)	apps/browser-guard/**
Deploy	Docker (multi-stage), docker compose, Helm (Postgres, Redis, Qdrant, Weaviate, HPA, Ingress)	deploy/** , docker-compose.yml
Tooling	Make targets, gotestsum, Playwright, newman, oxlint/oxfmt	Makefile

3. Repository layout

core/ Go module: the AI engine providers/<vendor>/ one package per provider (chat, responses, images, audio, video, batch...) schemas/ shared types (requests, responses, plugin interfaces, errors) mcp/ MCP client, code mode, credential store keyselectors/ weighted/random key selection strategies network/ HTTP client utilities (retries, timeouts, proxies) framework/ Shared infrastructure configstore/ ConfigStore + WorkspaceStore; tables/ has every GORM model logstore/ LLM/MCP/Browser AI log storage, aggregations, matviews, cleaners rbac/ Resource×Operation enforcement, route → requirement mapping encrypt/ AES encryption at rest for secrets in DB cluster/, loadbalancer/, routing/, circuitbreaker/, kvstore/, featureflags/ connectors/, objectstore/, vectorstore/, oauth2/, mailer/, alerts/, tracing/ migrator/, mysqlconn/, postgresconn/ DB drivers + schema migration helpers plugins/ telemetry, prompts, logging, governance, otel, semanticcache, compat, maxim, guardrails, connectors, modelcatalogresolver, mocker, jsonparser transports/unifai-http/ The server binary main.go flags, logger, version banner, bootstrap/start server/server.go UnifAIHTTPServer: bootstrap, middleware assembly, graceful shutdown server/plugins.go built-in plugin instantiation + ordering handlers/*.go every /api and /v1 handler (one file per feature area) integrations/*.go pass-through provider surfaces websocket/, ui/ WS pool/sessions; embedded UI assets ui/ React dashboard (Vite). pages under ui/app/workspace/** apps/browser-guard/ Python Guard agent + mitmproxy addon + installers + release artifacts configs/ config.json, mcp-library.json, model-parameters.json, pricing.json deploy/ docker/, helm/unifai/ docs/ this documentation set

4. Core module (core/)

4.1 Providers

Each directory under **core/providers/<vendor>** implements the shared provider contract from **core/schemas**. A provider package contains files such as **chat.go**, **responses.go**, **images.go**, **speech.go**, **transcription.go**, **videos.go**, **batch.go**, **models.go**, **errors.go**, **types.go**, **utils.go**. Capability differences that matter (from the files present):

Provider(s)	Notable capability files
openai	chat, responses, responseslifecycle, text, images, speech, transcription, videos, batch, files, embedding, realtime, websocket, large_payload, passthrough_usage
anthropic, gemini	chat, responses, count_tokens / models, embedding (gemini), types
vertex	chat, batch, cachedcontents, count_tokens, embedding, rerank, models
bedrock, bedrockmantle	chat, responses, embedding, images, batch
replicate, runware, runway	chat, images, videos, files
mistral	adds ocr.go on top of chat/transcription/cachedcontents
elevenlabs	audio-only
ollama, sgl, vllm, openrouter, parasail, opencode, groq, cerebras, deepseek, fireworks, huggingface, nebius, perplexity, xai, cohere	vendor subsets (mostly OpenAI-compatible shapes)
openaicompat	generic OpenAI-compatible provider + registry.go for many compatible endpoints
azure	Azure OpenAI, deployment-aware

Shared helpers live in **core/providers/utils**: **decompression.go**, **sse.go**, **stream.go**, **streamterminaldetector.go**, **pagination.go**, **images.go**, **audio.go**, **file.go**, **videos.go**,

bodysigner.go (request body signing for AWS-style auth), **modelparamscache.go**, **largerresponse.go**, **passthrough_stream.go**, **fetch.go**, **models.go**, **utils.go**.

4.2 Schemas

core/schemas holds the types shared by the gateway and every plugin: chat / responses / text / embedding / image / audio / video request-response types, **UnifAIConfig**, **UnifAIContext**, **ProviderConfig**, **NetworkConfig**, the plugin interfaces (**BasePlugin**, **LLMPlugin**, **MCPPlugin**, **ObservabilityPlugin**, **HTTPTransportPlugin**, **UnifAIHTTPMiddleware**), error types and log level/output enums.

4.3 MCP client

core/mcp implements the Model Context Protocol client used by the MCP Gateway: connection lifecycle, tool discovery, tool execution, per-user credentials (**credstore**), "code mode" (**codemode**) and utilities. The HTTP layer wraps it in **handlers/mcp.go** (clients, library, tool groups), **handlers/mcpserver.go**, **handlers/mcpsessions.go** and the OAuth2 set (**mcpoauth2.go**, **mcpoauth2discovery.go**, **mcpoauth2issuance.go**, **mcpoauth2consent.go**, **mcpoauth2sessions.go**, **mcpoauth2jwt.go**).

4.4 Key selection and networking

- **core/keyselectors** - weighted/random selection across multiple keys per provider.
- **core/network** - the outbound HTTP client for provider calls (proxy, TLS, timeouts, retries), driven by each provider's **NetworkConfig**.

5. Framework module (**framework/**)

5.1 ConfigStore

framework/configstore is the configuration database layer.

- Drivers: **postgres.go**, **mysql.go**, **sqlite.go**, **rdb.go**; interface in **store.go**; errors in **errors.go**.
- **migrations.go** plus **framework/migrator** (**addcolumn.go**, **dropcolumn.go**) handle schema evolution.
- **tables/** holds one file per domain: **user.go**, **workspace.go**, **enterprise.go**, **team.go**, **customer.go**, **virtualkey.go**, **key.go**, **provider.go**, **model.go**, **modelconfig.go**, **modelpricing.go**, **modelparameters.go**, **pricingoverride.go**, **budget.go**, **ratelimit.go**, **routingrules.go**, **mcp*.go**, **mcpoauth2*.go**, **prompts.go**, **promptVersions.go**, **promptSessions.go**, **skills.go**, **folders.go**, **featureflag.go**, **plugin.go**, **config.go**, **clientconfig.go**, **env.go**, **sessions.go**, **temptokens.go**, **dlock.go** (distributed locks), **logstore.go**, **vectorstore.go**, **smtp.go**, **confighash.go**.
 - **workspace.go** seeds the **RBAC catalog**, the three **system roles**, and exposes the resource/operation/name catalogs used by backend and UI.

- Secrets in the DB are encrypted with **framework/encrypt** when **UNIFAI_ENCRYPTION_KEY** is set (**tables/encryption.go**, **configstore/encryption.go**).

5.2 LogsStore

framework/logstore stores and aggregates request logs.

- Files: **logger.go**, **store.go**, **rdb.go**, **postgres.go**, **mysql.go**, **sqlite.go**, **hybrid.go**.
- **browser_ai.go** is the **entire Browser AI persistence layer** (see §9).
- Aggregations for dashboard/log pages: **matviews.go** (materialized views), **tables.go**, **cleaner.go** (retention), **asyncjob.go** (async inference jobs), **payload.go**.

5.3 RBAC (two enforcement layers, both applied to **/api/***)

1. **Resource × Operation** - **framework/rbac/enforce.go**, **PathRequirementFor(method, path)**

returns the requirement for a route (or **nil** = not RBAC-gated). **admin** resolves to *allow all*;

an unknown/unseeded role yields an **empty set = deny**. **View** and **Read** are interchangeable.

2. **Sidebar section grants** - **framework/rbac/sections.go**, **SectionRequirementFor(method, path)**,

used for section-scoped sessions. Keys are **parent** or **parent/child**; **parent/*** accepts the

parent or any child. Legacy **observability/browser-ai** is normalised to **browser-ai**.

Resource catalog (33): `GuardrailsConfig, GuardrailsProviders, GuardrailRules, UserProvisioning,

Cluster, Settings, Users, Logs, Observability, Dashboard, VirtualKeys, ModelProvider, Plugins,

MCPGateway, MCPToolGroups, MCPLogs, AdaptiveRouter, AuditLogs, Customers, Teams, RBAC, Governance,

RoutingRules, PromptRepository, PromptDeploymentStrategy, SkillsRepository, AccessProfiles, APIKeys,

Inference, Metrics, FeatureFlags, CircuitBreaker`

Operation catalog (6): **Read, View, Create, Update, Delete, Download**

System roles (auto-seeded in **configstore/workspace.go**, then merged on every boot so new

permissions are picked up - except for roles a customer has customised):

Role	DAC	Grants
admin	all-data	every permission
sub_admin	all-data	Read/View on Dashboard, Inference, Observability, MCPLogs, RoutingRules, Metrics ; full (Read/View/Create/Update/Delete/Download) on Logs - this is what gates all Browser AI write APIs; full on VirtualKeys, PromptRepository ; View/Read/Update on Governance ; View/Read/Create/Update on MCPGateway, MCPToolGroups ; View/Read on ModelProvider . No master provider keys, no system settings
user	own-data	Read/View only , on Dashboard, Logs, Inference, PromptRepository, Observability, MCPGateway (the readIDs set). The UI labels this "Prompt Repository only"; the backend also allows read-only views of logs/observability/MCP. Use allowed_sections to narrow what a user actually sees

Custom roles are created in the UI. **admin** is always re-synced to the full catalog.

*Browser AI note: every **/api/browser-ai/*** route maps to the **Logs** RBAC resource*

*(**PathRequirementFor**), so a role needs **Logs: View/Update** (or the **browser-ai** section grant)*

to see the Browser AI pages.

5.4 Other framework packages

Package	Purpose	API surface
cluster	multi-node config propagation	/api/cluster, /internal/cluster/kv
loadbalancer	adaptive routing across providers/keys	/api/load-balancer, /api/load-balancer/routes
routing	routing-rule engine	/api/governance/routing-rules, /api/routing-rules
circuitbreaker	failure isolation policies	/api/circuit-breaker/policies, /state
featureflags	server-side flags	/api/feature-flags
rbac, oauth2, temptoken	permissions, OAuth2 provider, short-lived tokens	/api/rbac/*, /api/scim/oauth/*
mailer, alerts	SMTP and alert delivery (webhooks)	/api/smtp-config, /api/alert-channels
connectors	external sinks (BigQuery, Kafka, Datadog, New Relic, PubSub)	/api/connectors
objectstore, vectorstore, kvstore	blob / vector / KV backends (Qdrant, Weaviate, Redis)	/api/vector-store-config, /api/cache
tracing	trace store + spans	/api/logs trace views
querscope, mcpcatalog, mcptoolgroups, mcp_headers, modelcatalog, encrypt, envutils	supporting services	-
streaming	SSE/stream utilities shared by transports	-

6. Plugins (plugins/)

A plugin implements one or more core interfaces and is loaded either as a **built-in** (by name) or as a **dynamic .so** via **SharedObjectPluginLoader (server/plugins.go)**.

Built-in load order (fixed placement indexes):

#	Plugin	Role
1	telemetry	request metrics/counters
2	prompts	prompt repository integration
3	logging	LLM/MCP log persistence
4	governance	virtual keys, budgets, rate limits, teams, customers, business units
5	otel	OpenTelemetry export
6	semanticcache	embedding-based response caching
7	compat	parameter compatibility/conversion between client and provider shapes

#	Plugin	Role
8	maxim	Maxim observability integration
-	guardrails	rule-based input/output validation + provider config
-	connectors	external log/analytics sinks
-	modelcatalogresolver	resolves model metadata/pricing
-	mock	mock provider for tests/E2E
-	jsonparser	tolerant JSON parsing helpers

Plugin config lives in **config.json** under **plugins**; plugins can be enabled/disabled and reordered at runtime (**GET/POST /api/plugins**). A plugin that fails to load is marked disabled and reported in the startup plugin-status table (**plugin status: <name> - <status>**).

7. HTTP surface (**transports/unifai-http**)

7.1 Entry point and flags

main.go:

Flag	Env fallback	Default
-port	APP_PORT	8001
-host	APP_HOST / UNIFAI_HOST	localhost
-app-dir	APP_DIR	./data
-log-level	LOG_LEVEL	info
-log-style	LOG_STYLE	json

Startup prints an ASCII banner (**-ldflags -X main.Version=...**), starts the pprof server, configures the logger, then **Bootstrap()** -> **Start()**.

7.2 Middleware assembly

Server handler wrapper (**server.go**):

```
SecurityHeadersMiddleware → CORS → RequestDecompression → router
```

API middleware chain (**apiMiddlewares**, applied to every **/api/*** route):

```
AuthMiddleware.APIMiddleware() session cookie / Bearer session token WorkspaceAuditMiddleware audits  
mutating /api/* into Audit Logs RBACMiddleware resource × operation + section grants
```

Inference middleware chain (**/v1/*** and pass-through):

```
TracingMiddleware → TransportInterceptor → AuthMiddleware.InferenceMiddleware → handler
```

TransportInterceptor runs **HTTPTransportPostHook** plugins and captures their log entries; the tracing defer runs afterwards so plugin logs land inside the trace.

7.3 Request identification on inference

- **Authorization: Bearer <virtual-key|provider-key>** or **x-api-key: <key>**.
- Dashboard-originated inference is enriched from the session cookie

(**enrichInferenceFromDashboardSession**); provider headers are normalised to a single

Authorization: Bearer ... (**x-api-key** is deleted).

- Password-as-Bearer (base64 **user:pass**) is **removed** - only session tokens are accepted as Bearer.
- Virtual Key self-service quota: **GET /api/governance/virtual-keys/quota** (the VK header is the credential).

7.4 Route groups

Group	Representative routes	Handler file(s)
Health / UI	GET /health , GET /robots.txt , / (SPA)	health.go , ui.go
Session / auth	/api/session/login , /logout , /register , /register/verify , /register/resend , /forgot-password , /verify-otp , /reset-password , /forgot-username , /is-auth-enabled , /ws-ticket , /api/session/users*	session.go , session_register_verify.go , auth_security.go
Workspace / RBAC	/api/users/{id}/role , /api/roles* , /api/permissions , /api/resources , /api/operations , /api/rbac/* , /api/access-profiles*	workspace.go , rbac.go , rbac_enforce.go , accessprofiles.go
Governance	/api/governance/virtual-keys* , /customers* , /teams* , /business-units* , /routing-rules* , /complexity-analyzer-config , /api/guardrails/*	governance.go , businessunits.go , team_members.go , guardrails.go
Inference	/v1/chat/completions , /v1/completions , /v1/responses* , /v1/embeddings , /v1/rerank , /v1/ocr , /v1/audio/* , /v1/images/* , /v1/videos* , /v1/batches* , /v1/files* , /v1/containers* , /v1/realtime , /v1/async*	inference.go , asynclineference.go , realtime_*.go , webrtc_realtime.go
Pass-through	/openai/* , /anthropic/* , /genai/* , /bedrock/* , /cohere/* , /cursor/* , /litellm/* , /langchain/* , /pydanticai/*	integrations/*.go
Observability	/api/logs* , /api/logs/dashboard , /api/logs/filterdata , /api/logs/rankings* , /api/logs/histogram/* , /api/mcp-logs* , /api/audit-logs , /api/connectors , /api/alert-channels	logging.go , auditlogs.go , alertchannels.go , integrations.go
Providers / models	/api/providers* , /api/keys* , /api/models* , /api/model-configs*	providers.go , provider_keys.go , list_models_vk.go
MCP	/api/mcp/clients , /api/mcp/library* , /api/mcp/client* , /api/mcp/tool-groups* , /api/mcp-logs , /api/mcp/inference	mcp.go , mcpserver.go , mcpsessions.go , mcptoolgroups.go , mcpheaders.go
Prompts / skills	/api/prompt-repo* , /api/prompt-deployments , /api/skills* , /api/skills/serve/*	prompts.go , prompts_lifecycle.go , skills.go , skills_serving.go
Config / platform	/api/config , /api/proxy-config , /api/vector-store-config , /api/smtp-config , /api/cache , /api/plugins , /api/feature-flags , /api/cluster , /api/circuit-breaker/* , /api/load-balancer*	config.go , plugins.go , featureflags.go , cluster.go , circuitbreaker.go , cache.go
Enterprise / SCIM	/api/scim/* , /api/scim/oauth/*	scim.go , scim_handlers.go , scim_oauth.go , scim_auth.go

Group	Representative routes	Handler file(s)
Browser AI	see §9	browser_ai*.go
WebSocket	/ws , WS tickets, responses/realtime WS	websocket.go, wsrealtime.go, wsresponses.go, ws_ticket.go
Dev	/api/dev/* + pprof (dev mode only)	devpprof*.go

7.5 Health and observability endpoints

- **GET /health** - pings ConfigStore, LogsStore and VectorStore concurrently (10 s timeout) and returns **{"status": "ok", "components": {"db_pings": "ok"}}**; returns HTTP 503 with the first failing component otherwise. Can be short-circuited by **client.disable_db_pings_in_health**.
- **GET /robots.txt** - static **Allow: /**.
- **GET /api/version** - build version.
- Logging skip list (no request logging): **/health, /_next, /api/dev**.
- Metrics (Prometheus) and traces come from the telemetry/otel plugins and the Logs API.
- Dev builds additionally expose pprof handlers (gated by **devpprof_prod.go** in release builds).

8. Data model (key tables)

8.1 ConfigStore tables

- **governance_users** - `id, username(uniq), email, password, role(admin|sub\_admin|user), status (pending|approved|rejected|email\_unverified|disabled), budget, rate\_limit, budget\_id, rate\_limit\_id, allowed\_prompt\_repos, allowed\_sections, reviewed\_at, external\_id, created\_at, updated\_at`.
- **IsApproved()** treats an empty status as approved (legacy rows).
- **rbac_roles** - `id, name(uniq), description, is\_system\_role, dac(all-data|own-data), permission\_ids(JSON), created\_at, updated\_at`.
- **access_profiles** - `id, name(uniq), description, is\_active, version, calendar\_aligned, tags(JSON), spec(JSON), created\_at, updated\_at` (reusable governance policy templates with activate / deactivate / clone operations).
- **governance_business_units** - **id, name(uniq), team_ids(JSON), budget(JSON), rate_limit(JSON)**.
- Grouped in **tables/workspace.go**: **alert_channels, circuit_breaker_policies, mcp_tool_groups, prompt_deployments, workspace_settings, virtual_key_users, virtual_key_teams, virtual_key_customers, audit_logs**.
- Plus the core governance tables: teams, customers, virtual keys, provider keys, models, model configs, pricing + pricing overrides, budgets, rate limits, routing rules, prompts/versions/sessions, skills, folders, feature flags, plugins, sessions, temp tokens, distributed locks, SMTP, vector-store config, MCP clients/library/OAuth2 issuance.

8.2 Browser AI tables (**framework/logstore/browser_ai.go**)

Table	Purpose	Notable columns
browser_ai_logs	one row per intercepted prompt / upload	id, timestamp, platform, domain(host), user_prompt_full, action, status, rule_triggered, risk_score, predictive_risk, agent_id, agent_hostname, client_ip, reply_bot_text, attachment_name/type/size/path/expiry, metadata(JSON)
browser_ai_search_logs	search-engine queries + clicked results	id, timestamp, engine, browser, is_incognito, query, clicked_url, clicked_title, url, host, agent_id, predictive_risk
browser_guard_rules	guard rules	id, name, pattern, rule_type(regex or ai_bot), severity, action, warning_message, active, description, bot_provider, bot_model, bot_prompt, bot_reference_image, bot_reference_image_type
browser_ai_target_websites	monitored AI sites	id, domain(uniq), platform_name, monitored, block_entire_website, host_role(ui/chat/file), status(MONITORED/PAUSED/BLOCKED), parent_id
browser_ai_controls_settings	single-row interaction policy (browser-controls-default)	upload/download warning text and policy toggles
browser_ai_agents	Guard fleet inventory	id, hostname, agent_type(endpoint/network), status(active/uninstall_pending/uninstalled), last_seen_at, health_detail, contact_email(+pinned), uninstall_key_hash, uninstall_key_enc, has_uninstall_key, uninstall_key_rotated_at, proxy_bundle_sha
browser_ai_agent_settings	company-level uninstall key	id, uninstall_key_enc
browser_guard_fleet_config	company Guard defaults that agents pull on heartbeat	id, ... notes
browser_guard_rebuild_logs	Guard package rebuild history	id, status(success/failed), bundle_sha, message, log, created_at
browser_ai_warning_emails	employee warning e-mail log	id, sent_at, to_email, message, agent_id, error_detail

- Attachment bytes stay on disk (**APP_DIR/attachments**); the DB stores size, path and expiry metadata.
- **BrowserAIManager** (same file) implements every CRUD/aggregation API used by the handlers,

including **GuardSeverityScore** (severity -> predictive risk score + label) and

NormalizeBrowserAIAgentType.

9. Browser AI (UnifAI Guard) - technical detail

9.1 Backend routes (**handlers/browser_ai.go** registers all of these)

Logs / insights

```
GET /api/browser-ai/logs list + filter prompt logs GET /api/browser-ai/logs/stats aggregated stats DELETE
/api/browser-ai/logs delete all POST /api/browser-ai/logs/bulk-delete DELETE /api/browser-ai/logs/{id} GET
/api/browser-ai/search-logs POST /api/browser-ai/search-logs agent ingest DELETE /api/browser-ai/search-logs
POST /api/browser-ai/search-logs/bulk-delete DELETE /api/browser-ai/search-logs/{id} GET
/api/browser-ai/insights/stats Guard Insights
```

Rules / controls / targets

```
GET|POST /api/browser-ai/rules + POST /rules/import, PUT|DELETE /rules/{id} POST
/api/browser-ai/rules/test-bot test an AI Guard Bot rule POST /api/browser-ai/rules/generate-regex generate
regex from a policy description GET|PUT /api/browser-ai/controls interaction controls (single row) GET
/api/browser-ai/targets + POST /targets, POST /targets/import, PUT|DELETE /targets/{id}
```

Fleet / agents / setup

```
GET /api/browser-ai/agents list agents (filters: status, source, search) POST
/api/browser-ai/agents/heartbeat agent registration + health (Guard secret) POST
/api/browser-ai/agents/bulk-delete DELETE /api/browser-ai/agents/{id} PUT
/api/browser-ai/agents/{id}/contact-email GET /api/browser-ai/agents/settings GET|PUT
/api/browser-ai/agents/uninstall-key company uninstall key GET /api/browser-ai/agents/{id}/uninstall-key
POST /api/browser-ai/agents/{id}/uninstall-key/rotate POST /api/browser-ai/agents/uninstall-verify employee
key check POST /api/browser-ai/agents/uninstall agent requests removal POST
/api/browser-ai/agents/uninstall-ack POST /api/browser-ai/agents/uninstall-status POST
/api/browser-ai/agents/{id}/remote-uninstall admin-initiated removal GET|PUT /api/browser-ai/fleet-config
company-wide Guard defaults GET /api/browser-ai/setup/info GET /api/browser-ai/setup/download.zip aliases:
-windows.zip, -mac.zip POST /api/browser-ai/setup/rebuild rebuild + republish Guard packages GET
/api/browser-ai/setup/rebuild-history GET /api/browser-ai/setup/proxy-bundle.json GET
/api/browser-ai/setup/proxy-bundle.zip hot-update bundle for installed Guards POST
/api/browser-ai/send-warning-email
```

Ingest / PAC / helpers

```
POST /api/browser-ai/intercept agent prompt/upload event POST /api/browser-ai/intercept-file agent file
event GET /api/browser-ai/attachments/{id} attachment download (auth-gated) GET /api/browser-ai/proxy.pac
PAC script GET /api/browser-ai/pac PAC script (alias used by GPO / network proxy) GET
/api/browser-ai/ollama-models models available for AI Guard Bot rules
```

9.2 Guard-fleet authentication (no user session)

isPublicBrowserAIRoute and **isGuardKeyBrowserAIRoute** (**handlers/middlewares.go**) let the Guard agent call the backend **without a dashboard session cookie**:

- **Public (secret-validated):** **intercept**, **intercept-file**, **search-logs** (POST), **proxy.pac**, **pac**, **agents/heartbeat**, **agents/uninstall-verify**, **agents/uninstall**, **agents/uninstall-ack**, **agents/uninstall-status**.
- **Guard-key:** **targets**, **rules**, **controls**, **fleet-config** (GET), **setup/download*.zip**, **setup/proxy-bundle.json**, **setup/proxy-bundle.zip**.
- The fleet secret is **UNIFAI_GUARD_SECRET**. With **UNIFAI_GUARD_REQUIRE_SECRET=1** (production default in **.env.example** and compose) agent APIs **fail closed** when the secret is missing or wrong (**browser_ai_guard_security.go**).
- **UNIFAI_PAC_ALLOW_QUERY_PROXY=0** (default) makes the PAC ignore a client-supplied **?proxy=** and use only fleet/env values - anti PAC-hijack.

9.3 Guard desktop agent (apps/browser-guard/agent)

unifai_agent.py is the entry point. Supporting modules:

Module	Responsibility
guard_bootstrap.py	single-instance lock, logging bootstrap, runtime version resolution
agent_config.py	env -> JSON config precedence (UNIFAI_BACKEND_URL , UNIFAI_PROXY_ADDR , UNIFAI_PAC_URL , UNIFAI_PAC_SYNC_SECONDS , UNIFAI_SERVER_MODE , UNIFAI_AGENT_TYPE , UNIFAI_LISTEN_HOST , UNIFAI_PAC_ADVERTISE_ADDR , UNIFAI_AGENT_ID , UNIFAI_AGENT_HOSTNAME)
agent_identity.py	persistent agent ID + host/OS metadata for the fleet inventory
agent_heartbeat.py	periodic heartbeat, fleet config pull, remote-uninstall intent, health detail
agent_health.py	port checks (port_open , free_proxy_port), health loop
agent_pac_content.py	backend reachability check, PAC fetch, local PAC write
agent_pac_server.py	local PAC/health HTTP server (PAC_HTTP_PORT , default 18195, falls back to +1/+2 if busy)
agent_pac_orchestration.py	apply PAC with cache-bust, clear/restore runtime, fail-open direct mode
agent_proxy_engine.py	runs the mitmproxy worker child process
agent_proxy_bundle.py	SHA-tracked hot-update bundle, mark-bad, relaunch, running SHA
agent_certs.py	installs the mitmproxy CA into the OS trust store, writes ca_install_status.txt
agent_browser_policy.py	browser QUIC / DoH policy adjustments
agent_autoupdate.py	compares UNIFAI_GUARD_RUNTIME_VERSION with the installer version and updates
agent_lifecycle.py	first-run prompt, uninstall flow (key-gated), user messages
agent_logging.py	file logging, get_resource_path
agent_http.py , agent_state.py , guard_platform.py	HTTP client, local state, OS paths (data_dir , log_hint_path , register_autostart)

Default paths: data %LOCALAPPDATA%\UnifAI\Guard (Windows) /

~/Library/Application Support/UnifAI/Guard (macOS); log **unifai_guard.log**;

CA status **ca_install_status.txt**.

9.4 mitmproxy addon (apps/browser-guard/proxy/browser_ai_proxy.py)

The addon is **not** a normal Python package. **unifai_proxy_parts/MANIFEST.txt** lists the parts that are loaded in order into one shared namespace (preserving the original single-file monolith behaviour):

```
config_caches_rules.py config, caches, target/rule fetch + matching helpers_prompts.py prompt extraction +
helper utilities uploads_detect.py upload / file detection file_policy.py per-file-type policy decisions
extract_office_backend.py Office/PDF/image text extraction + backend calls responses_inject.py response
rewriting / block injection (SSE aware) responses_addon.py response learning (e.g. filename discovery)
```

Flow: target-domain match -> extract prompt/attachment -> evaluate rules (regex locally; **ai_bot** via the

configured LLM/Ollama) -> verdict **Allowed | Redacted | Warned | Blocked | SiteBlocked | Bot Answered**

-> inject the browser-visible response (JSON error, Anthropic SSE **unifai_guard_blocked**, etc.)

-> POST the event to **/api/browser-ai/intercept**.

Editing any part requires rebuilding the Guard (**installer/build_installer.bat** on Windows, **installer/build_macos.sh** on macOS) and keeping cross-part names intact.

9.5 Guard distribution and hot update

- Build: **make build-guard-windows** (Inno Setup) / **make build-guard-mac** (+ **-pkg**) - see **apps/browser-guard/README.md**. Without **make**, run **installer\build_installer.bat**.
- Config sync: **python apps/browser-guard/scripts/sync_config_from_env.py** regenerates **config/unifai_guard_config.json** and the docs from **.env** (**SERVER_DOMAIN**, **UNIFAI_PROXY_ADDR**, **PAC_HTTP_PORT**, **UNIFAI_GUARD_SECRET**).
- Server-side artifacts live in **apps/browser-guard/release/** and are copied into the image (**/app/release**), so **Browser AI -> Setup -> Download Setup ZIP** serves them.
- **Browser AI -> Setup -> Rebuild & Publish** re-publishes the proxy/agent Python sources as a bundle (**/api/browser-ai/setup/proxy-bundle.zip**); installed agents pick it up via **agent_proxy_bundle.py**.
- Version file: **apps/browser-guard/release/VERSION.txt** -> **1.1.15**.

10. Frontend (ui/)

10.1 Build and integration

- Vite + React 19 + TanStack Router (file-based routes under **ui/app/****, generated into **ui/app/routeTree.gen.ts**).
- State: Redux Toolkit with RTK Query APIs under **ui/app/_fallbacks/enterprise/lib/store/apis/*** (**accessProfileApi**, **alertChannelsApi**, **auditLogsApi**, **businessUnitsApi**, **circuitBreakerApi**, **clusterApi**, **connectorsApi**, **loadBalancerApi**, **mcpToolGroupsApi**, **promptDeploymentsApi**, **rbacApi**, **scimApi**, **virtualKeyUsersApi**, ...).
- **npm run build:unifai** = rebrand + **vite build** + typecheck; **npm run copy-build** copies **out/** into **transports/unifai-http/ui**, which the Go binary embeds (**//go:embed**). The UI therefore ships **inside** the server binary - no separate web server is required.
- Dev server: **npm run dev**; **make dev** runs UI + API together.
- Quality gates: **npm run typecheck** (tsc --noEmit), **npm run lint** (oxlint), **npm run format** (oxfmt).

10.2 Pages

Top level: **/** (landing), **/login**, **/signup**, **/oauth/consent**, **/pprof**, **/workspace/****, plus **__root**, **__notFound**, **__error**.

Workspace pages (one **page.tsx** per route, each with a **layout.tsx**):

Section	Routes
Observability	dashboard, logs, logs/connectors, mcp-logs, observability, audit-logs, alert-channels
Browser AI	browser-ai (tabs: overview, logs, search-logs, rules, targets, agents, telemetry, setup)
Models	providers, providers/model-limits, providers/routing-rules, model-catalog, model-limits, custom-pricing, custom-pricing/overrides, complexity-router, routing-rules, routing-rules/tree, circuit-breaker
MCP Gateway	mcp-gateway, mcp-registry, mcp-registry/library, mcp-registry/oauth-callback, mcp-tool-groups, mcp-sessions (+ auth, auth-failed, auth-success), oauth-grants, mcp-auth-config, mcp-settings
Governance	governance, governance/virtual-keys, governance/users, governance/teams, governance/business-units, governance/customers, governance/rbac, governance/access-profiles, rbac, virtual-keys, scim, scim/oauth-discover-callback
Guardrails	guardrails, guardrails/configuration, guardrails/providers
Platform	plugins, cluster, adaptive-routing (+ settings), prompt-repo (+ prompts), skills-repo, docs, config (+ api-keys, caching, client-settings, compatibility, feature-flags, logging, mcp-gateway, observability, performance-tuning, pricing-config, proxy, security)

10.3 Sidebar and section gating

ui/components/sidebar.tsx builds the menu from **ui/lib/constants/workspaceSections.ts**:

```
observability | browser-ai | models | mcp-gateway | plugins | governance | guardrails | cluster-config |
adaptive-routing | prompt-repository | skills-repository | settings
```

Each section has a **defaultPath** and optional child **items**. Visibility is decided by:

1. RBAC hooks - **useRbac(RbacResource.X, RbacOperation.View)**.
2. Role checks for admin-only pages (Users requires **admin** or **sub_admin**).
3. Section grants for section-scoped sessions (**scopedSidebarSections** from **authStatus.allowed_sections**), with automatic redirect to **getDefaultPathForSections(...)**

when the user lands on a route they are not granted.

Browser AI is reachable with **Logs** view RBAC or the **admin/sub_admin** role

(**hasBrowserAiAccess**), matching the backend mapping of **/api/browser-ai/* -> Logs**.

10.4 Browser AI UI implementation files

ui/app/workspace/browser-ai/: **page.tsx** (tabs, tables, dialogs), **browserAiTypes.ts**, **browserAiConstants.ts** (**GUARD_BOT_OLLAMA_PROVIDER = "ollama"**, **GUARD_BOT_OLLAMA_MODEL = "llama3.2"**, reference image max 512 KB), **browserAiFormat.ts**, **browserAiLogHelpers.ts**, **browserAiGuardHelpers.ts**, **guardBotModelPickers.tsx**, **guardRuleAIEvaluatorFields.tsx**, **guardRuleImportDialog.tsx**, **targetImportDialog.tsx**, **regexLiveTestPanel.tsx**, **logPromptPreviewCell.tsx**, **logTimestampCell.tsx**, **logBadges.tsx**, **attachmentPreview.ts**, **relatedHosts.ts**, plus import templates under **examples/**.

Types as implemented: **GuardRuleAction = "BLOCK" | "REDACT" | "WARN"**,

GuardRuleSeverity = "CRITICAL" | "HIGH" | "MEDIUM", GuardRuleEvalMode = "ai" | "regex".

11. Security model

Area	Implementation
Dashboard sessions	Session rows stored in DB; token cookie with HttpOnly + SameSite=Lax (+ Secure only for real TLS or a trusted proxy sending X-Forwarded-Proto: https); default 24 h TTL
WebSocket auth	Cookie-only (/api/session/ws-ticket issues short-lived tickets); legacy ?token= query auth removed (token-leak risk)
Inference auth	Virtual Key or provider key via Authorization: Bearer / x-api-key ; enforce_auth_on_inference flag
Brute force	Per-IP: 30 failed logins / 15 min window; per-username lockout rows in the DB; remaining minutes returned to the client
Password reset	PASSWORD_RESET_SECRET (min 32 chars) HMAC-signed token, 10 min expiry; 5 requests/hour/target + 10/hour/IP; OTP max 5 attempts; 2 min cooldown
Registration	Self-signup with e-mailed code (email_unverified) -> admin approval (pending -> approved) or rejected ; SCIM can set disabled
Authorization	Two RBAC layers (resource×operation, section grants) + audit middleware recording mutating /api/* actions
Secrets at rest	framework/encrypt encryption for DB-stored secrets when UNIFAI_ENCRYPTION_KEY is set (provider keys, alert channels, uninstall keys)
Guard fleet	Shared UNIFAI_GUARD_SECRET , fail-closed with UNIFAI_GUARD_REQUIRE_SECRET=1 ; PAC hijack protection via UNIFAI_PAC_ALLOW_QUERY_PROXY=0
Guard uninstall	Company and per-agent uninstall keys, stored hashed + encrypted, rotatable, with an audit trail and remote-uninstall flow
Transport	TLS terminated at the edge/reverse proxy; TRUST_PROXY_HEADERS=1 honours X-Forwarded-Proto/X-Forwarded-For only from trusted peers (TRUSTED_PROXIES required for public edges)
CORS	client.allowed_origins (from SERVER_DOMAIN); optional ALLOW_LOCALHOST_CORS=1 for a local UI against a remote API
Headers	SecurityHeadersMiddleware applied to every response
Open bootstrap	When auth is disabled, unauthenticated admin is allowed only from loopback or with ALLOW_OPEN_AUTH=1 (local bootstrap only)
Inference hardening	Provider headers normalised (x-api-key removed), password-as-Bearer removed, error sanitisation (errorsanitizer.go)
Destructive ops	Admin-only guards (adminOnlyLogs , requireGuardAdmin); attachment access privileged

12. Build, test and developer workflow

12.1 Gateway

```
make setup-workspace # go work use ./core ./framework ./plugins/... ./transports make dev # UI (vite) + API
with hot reload (air) make build # build UI then the Go binary make run # build + run make fmt ; make lint #
gofmt / linter make test # unifai-http transport tests (gotestsum) make test-core # core provider tests
(PROVIDER=openai TESTCASE=... PATTERN=...) make test-framework # framework tests make test-plugins # plugin tests
make test-http-transport # HTTP transport tests make test-governance # governance tests make test-mcp # MCP
tests (TYPE=... TESTCASE=...) make test-semantic-cache # semantic cache e2e make test-all # all of the above make
run-e2e ; make run-e2e-ui ; make run-e2e-headed ; make run-e2e-api make test-integrations-py ; make
test-integrations-ts make run-provider-harness-test ; make run-cli-harness-test
```

Reports: **make generate-html-reports**; cleanup with **make clean-test-reports**.

12.2 UI

```
cd ui npm ci npm run dev # Vite dev server npm run typecheck # tsc --noEmit npm run lint # oxlint npm run
build # vite build + typecheck + copy into transports/unifai-http/ui
```

12.3 Guard

```
make sync-guard-config # regenerate Guard config/docs from .env make build-guard-windows # Windows: Setup.exe
+ Windows ZIP (Inno Setup 6 required) make build-guard-mac # macOS: .app + macOS ZIP (must run on a Mac) make
build-guard-mac-pkg # macOS: .pkg from the existing .app make help-guard # list Guard targets
```

Guard unit tests live in **apps/browser-guard/agent/tests/** and **apps/browser-guard/proxy/tests/**

(PAC targets, proxy bundle, file-type extraction, latency, rule actions, target predict, upload names).

12.4 Docker / Helm

```
make docker-image # LOCAL=1 uses deploy/docker/Dockerfile.local (go workspace based) make docker-run # run
the container (CONFIG=... optional) make helm-index # repackage chart + regenerate deploy/helm/index.yaml
```

12.5 Conventions

- Go modules resolve through **go.work**; add new modules with **go work use**.
- Handlers keep the **RegisterRoutes(router, middlewares...)** pattern and are grouped by feature.
- UI files use camelCase names, Radix primitives from **components/ui**, and RTK Query for API calls.
- Guard proxy parts must stay in **MANIFEST.txt** order.

13. Extension points

Goal	Where to work
Add an LLM provider	new package under core/providers/<name> implementing the provider contract, then register it in the provider registry
Add a pass-through API surface	new file in transports/unifai-http/integrations/ + registration in integrations/router.go
Add a plugin	new module under plugins/<name> exposing PluginName ; add a case in loadBuiltinPlugin (server/plugins.go) and add the module to go.work
Add a dashboard API	new handlers/<feature>.go with RegisterRoutes , then call it from server.RegisterAPIRoutes
Gate a new route with RBAC	extend PathRequirementFor (framework/rbac/enforce.go) and, for section-scoped sessions, SectionRequirementFor (framework/rbac/sections.go)

Goal	Where to work
Add an RBAC resource	add it to RBACResourceNames (framework/configstore/workspace.go); permissions are generated from the cartesian catalog
Add a UI page	create ui/app/workspace/<route>/page.tsx + layout.tsx , add the section item in ui/lib/constants/workspaceSections.ts , and the nav entry in ui/components/sidebar.tsx
Add a Guard detection	edit the relevant part in apps/browser-guard/proxy/unifai_proxy_parts/ (keep MANIFEST order) and rebuild the Guard
Add a Guard rule type	BrowserGuardRule (framework/logstore/browser_ai.go) + the UI rule editor + the proxy evaluator
Change gateway guardrail behaviour	plugins/guardrails + /api/guardrails/*

14. Glossary

Term	Meaning in this codebase
Virtual Key (VK)	A gateway-issued credential that maps to budgets/limits/teams/customers and is used by inference clients
ConfigStore	Config/workspace database (users, roles, keys, providers, settings)
LogsStore	Log database (LLM logs, MCP logs, Browser AI logs, aggregates)
Section grant	Sidebar-section permission (governance/users) used for section-scoped sessions
DAC	Data access scope on a role: all-data or own-data
Target website	A monitored AI domain (with platform name, host role and monitored/blocked flags)
Guard rule	A detection rule: regex (pattern) or ai_bot (LLM-judged, optional reference image)
Guard agent	A Guard installation reporting to the fleet (endpoint or network)
Fleet config	Company-wide Guard defaults that agents pull on heartbeat
PAC	Proxy auto-config script served by the backend, pointing browsers at the Guard proxy
Proxy bundle	Server-published hot-update bundle of the Guard proxy/agent Python code
Uninstall key	Company/per-agent secret required to turn the Guard off
Intercept	The Guard -> backend event that records a prompt, upload or verdict